

Beyond Zero-Cost: Understanding Rust Safety Overheads in Systems Code

Soham Bagchi
University of Utah

Manvik Nanda
University of Utah

Anton Burtsev
University of Utah

Abstract

Rust is increasingly used as a viable alternative to C and C++ for systems development. A central appeal of Rust is its “zero-cost” abstractions and, specifically, its ability to enforce safety without the overhead of garbage collection. In practice, however, only a subset of safety properties can be enforced statically, and the runtime cost of Rust safety remains poorly understood.

In this paper, we take an in-depth look at the overheads of Rust safety using the example of a fast network interface driver, one of the most performance-critical components of the modern systems stack. We develop several semantically equivalent C and Rust implementations of an Intel E810 100 GbE device driver. Our analysis shows that Rust can approach the performance of C when the code is lean, closely follows C idioms, and enforces nearly all safety properties statically. At a batch size of 64 packets, a carefully optimized safe Rust implementation achieves 93% of the throughput of unsafe C while executing approximately 12% more instructions per packet. Its only sources of runtime safety overhead are null-pointer and bounds checks. Adding overflow checking and explicit error handling to avoid panics reduces throughput by a further 7% and 10%, respectively, relative to the basic safe Rust implementation. Even seemingly simple abstractions and representation changes, such as widening a pointer representation, alter the generated machine code, increasing register pressure and introducing additional memory accesses and control instructions.

CCS Concepts: • Software and its engineering → Software safety; Software performance.

ACM Reference Format:

Soham Bagchi, Manvik Nanda, and Anton Burtsev. 2026. **Beyond Zero-Cost: Understanding Rust Safety Overheads in Systems Code**. In *Workshop on Programming Languages and Operating Systems (PLOS '26)*, September 29–October 2, 2026, Prague, Czech Republic. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3831586.3838147>



This work is licensed under a Creative Commons Attribution 4.0 International License.

PLOS '26, September 29–October 2, 2026, Prague, Czech Republic

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2877-8/26/09

<https://doi.org/10.1145/3831586.3838147>

1 Introduction

Over the last decade, Rust has emerged as a viable alternative to C and C++ for low-level systems programming. A central appeal of Rust is the ability to enforce safety without a managed runtime, and specifically without garbage collection [10]. Several studies estimate that adopting a memory-safe language such as Rust can eliminate close to 70% of low-level systems vulnerabilities associated with language safety [12, 13, 17, 8]. These vulnerabilities include out-of-bounds accesses, lifetime violations such as use-after-free and double-free, uninitialized memory accesses, invalid pointer dereferences, integer overflows and underflows, and data races.

Recognizing this combination of security benefits and practical performance, many projects have adopted Rust for systems development. Linux, FreeBSD, and, more recently, Windows have introduced support for Rust as a way of developing kernel extensions and device drivers [25, 23, 27]. Cloud infrastructure projects rely on Rust to implement security-critical virtual-machine monitors, paravisors, and container runtimes [7, 22, 4, 15, 16, 9]. High-performance infrastructure projects such as Cloudflare’s Pingora, the Wasmtime WebAssembly runtime, rustls, and Apache DataFusion illustrate that Rust can be used in latency-sensitive networking, cryptography, and analytics systems [5, 3, 21, 1].

Through a combination of compile-time analysis and runtime checks, Rust enforces safety. Specifically, an ownership and borrowing discipline that controls aliasing and mutation of values and, most importantly, allows compile-time reasoning about object lifetimes, eliminating the need for garbage collection. In Rust, references are compiled much like ordinary machine pointers, and lifetimes have no runtime representation, i.e., moving a value or creating a borrow does not result in runtime reference counting.

However, not all safety properties can be enforced at compile time. Rust statically enforces ownership, lifetime, borrowing, and trait-bound constraints, including thread-safety constraints expressed through the `Send` and `Sync` traits. Properties that depend on runtime values instead require dynamic checks, additional state, or different data representations. For example, indexing arrays and slices introduces bounds checks; `take()` replaces `Some(T)` with `None`; enums without a suitable “niche” may require explicit discriminants; and dynamically sized types use fat pointers carrying metadata such as a slice length or vtable pointer.

We argue, however, that the performance overheads of

Rust safety mechanisms are not well understood. On the one hand, a range of academic work suggests that Rust introduces nontrivial overhead. NetBricks reports near-C performance for simple DPDK-based network functions, but also observes overheads from bounds checks in array-heavy code and a 14% slowdown for a Rust longest-prefix-match implementation [20]. The ixy high-level driver study finds that its Rust driver executes substantially more instructions per packet than C while suffering only a small throughput loss [6]. A broader Rust-versus-C microbenchmark study reports that Rust is slower on its algorithm benchmarks and attributes a large part of the measured overhead to compiler-inserted runtime checks, with bounds-checking accounting for 52.7% of the overhead [28]. The Rust-for-Linux study reports that driver performance varies across implementations and identifies checks, layout differences, sharing patterns, and missing low-level optimizations as possible causes [11]. On the other hand, the Rust Performance Book argues that many runtime checks can be eliminated with careful code structure [19], that, if used correctly, **Rust can be fast**.

In this paper, we take an in-depth look at the performance overheads of Rust safety mechanisms and how they affect the fastest parts of systems code. Specifically, we ask: how much overhead does Rust introduce in the critical path of a modern high-throughput device driver with a tight cycle budget? We develop several functionally equivalent device drivers in C and Rust for the Intel E810 100 GbE adapter.

All driver variants were developed as user-level processes that directly access the device via device passthrough, factoring out overheads such as kernel and framework abstractions. They also use optimizations typical of modern packet-processing frameworks: no allocation or synchronization on the critical path, packet-prefetching, and huge pages.

Our analysis shows that Rust can approach the performance of C when the code uses lean, C-like data structures and avoids unnecessary representation overhead. A carefully optimized safe Rust implementation (`Rust|Safe(RefPool)`, Section 4) executes approximately 12% more instructions per packet than unsafe C (129 versus 115) while achieving 93% of its throughput. Its runtime safety overhead is limited to five bounds checks and two null-pointer checks not present in C. Additional safety features introduce further costs. Overflow checking and explicit error handling to avoid panics reduce throughput by a further 7% and 10%, respectively, relative to the basic safe Rust implementation. Adding `Result<T>` to avoid panics inhibited inlining hence, increasing store traffic and reducing throughput (`Rust|Safe(RefPool+Errors)`, Section 4).

The cost of Rust safety is not explained by bounds checks alone. At a batch size of 256, our regression model associates each additional instruction with approximately 0.063 cycles per packet. The model assigns each store an additional premium of 0.283 cycles, for a modeled total cost of approximately 0.35 cycles per packet. A typical bounds check adds three non-store instructions—a load, a comparison, and

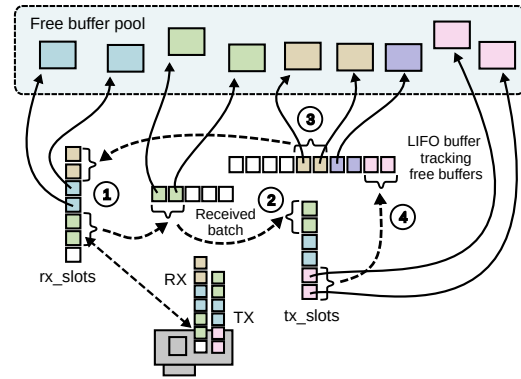


Figure 1. L2 forwarding path and packet management

a branch (Figure 2)—making it relatively lightweight. By contrast, the functions returning `Result<T>` in `Rust|Safe(RefPool+Errors)` were not inlined, introducing additional stores and a significant performance penalty.

In more complex Rust code, safety mechanisms can have compounding effects. Representation and compiler decisions can increase the numbers of loads, stores, comparisons, and branches, as well as register pressure and spills. For example, a more complex safe Rust implementation that maintains a metadata array for each packet and uses a widened 16-byte buffer reference has a modeled store cost of approximately 0.73 cycles per packet. Our analysis attributes this higher cost to increased store-buffer pressure.

At a high level, our performance recommendation is straightforward: use lean Rust code that follows C idioms and employs compact representations that preserve niche optimization. For such code, the remaining runtime safety overhead is primarily due to bounds checks and checks of optional pointer values.

Our study is limited to one device driver and one hardware platform. Although we believe that our E810 driver is representative of a high-throughput I/O path, broader studies are needed to determine how widely these results generalize. We hope that this work spurs further research on compiler and microarchitectural support for safe systems code. Our results specifically motivate better compiler inlining of functions returning `Result<T>` and further investigation into whether greater store-buffer and register capacity can improve the performance of safe systems workloads.

2 Background: Packet Forwarding

To understand the overheads of Rust, we study a fast user-level, DPDK-style device driver for the Intel E810 100 GbE NIC. We implement several semantically equivalent versions of the driver in C and in safe and unsafe Rust, keeping them as close as possible: the same data structures, the same code blocks when possible, and the same error checks. The versions, however, do differ in how they manage references to packet buffers (e.g., unsafe pointers, safe references, or indices), whether they enable bounds and overflow checks,

and whether they avoid panics.

At a high level, all versions implement a minimal buffer-management pattern used by high-speed poll-mode drivers such as DPDK and ixy [6]. A preallocated per-core pool of packet buffers with a lightweight buffer allocator selects the next available buffer reference from a LIFO free buffer list.

On the fast path (Figure 1), the application thread polls a receive (RX) queue for a bounded batch of completed DMA descriptors (①). We maintain two shadow arrays, `rx_slots` and `tx_slots`, that correspond to individual entries in the receive (RX) and transmit (TX) DMA descriptor rings. These arrays preserve each buffer address because the hardware overwrites the address field with status information after processing the descriptor. For each received packet, the driver uses the descriptor index (position in the descriptor ring) to look up the corresponding packet buffer in the slot array, records the received length, replaces the corresponding `rx_slots` entry and RX DMA descriptor with a fresh buffer from the packet allocator pool (③), rewrites the layer-2 header with the forwarding information, and submits the original received buffer directly to the corresponding TX queue, tracking the packet in `tx_slots` while it is in flight in the TX DMA descriptor ring (②). The TX completion logic polls for completed packets and returns transmitted buffers to the free pool (④).

3 Rust Safety

To enforce safety, Rust assigns each value a unique owner and enforces an ownership discipline to regulate how that value may be accessed through references. In particular, Rust ensures that a value may either have multiple shared immutable references or one exclusive mutable reference, but not both at the same time. Combined with lexical lifetimes, this ownership discipline lets the compiler reason about when values are live, when references remain valid, and when resources can be deterministically released as values go out of scope. Type checking, trait bounds, and the `Send` and `Sync` marker traits constrain how values can be transferred and shared across threads. Together, these mechanisms allow the compiler to reject a range of low-level errors common in unsafe languages, including use-after-free, double-free, uninitialized access, dangling references, iterator invalidation, and data races.

However, not all safety conditions can be enforced statically; some instead require runtime checks and state to enforce access bounds and ownership rules.

Bounds checks for indexing and slicing In most cases, the bounds of arrays and slices are not known statically. To enforce spatial safety, Rust inserts a runtime check in front of the access. Internally, Rust lowers each safe array or slice access into an explicit proof obligation: the index must be smaller than the length of the indexed object. An access like `slice[i]` is represented in MIR as a bounds-check assertion similar to `assert(i < slice.len())`. If the assertion cannot be

```
1 let old_buf = std::mem::replace(&mut self.tx_slots[idx as
    usize], Some(buf.into_in_flight()))
2 .expect("missing tx parked slot");

(a) Safe Rust

1 mov rsi,QWORD PTR [rsp+0xe48] // bounds: load tx_slots.len()
2 cmp rsi,rdi                  // bounds: idx < len ?
3 jbe 1fac2                     // bounds: panic
4 mov rax,QWORD PTR [rsp+0xe40] // reload slots base
5 mov r13,QWORD PTR [rax+rdi*8] // replace(): read old slot
6 mov QWORD PTR [rax+rdi*8],rbx // replace(): write new slot
7 test r13,r13                  // .expect(): None test
8 je 1face                      // .expect(): panic if None

(b) Intel assembly
```

Figure 2. Safe Rust and assembly code for a `tx_slots` buffer update. Rust safety adds three instructions (L1-3) for the bounds check and two instructions to check for `None` and branch to the panic path (L7-8).

discharged, code generation emits a conditional branch to Rust’s panic path before performing the memory access.

For example, in our code, the access to the `tx_slots` array that tracks packet buffers added to device DMA descriptors in our E810 driver (Figure 2a) results in a bounds check (Figure 2b, L1-3) in front of the access (L5-6), which triggers a panic (L3). Line 4 reloads the base of the slot array from a temporary location on the stack into the `rax` register, an example of register pressure.

Bounds checks might also seem unavoidable in an unsafe language such as C—after all, most indices are dynamic, e.g., depend on the number of received packets, dynamically configured buffer sizes, etc. In practice, unsafe code relies on high-level semantic guarantees to avoid naive bounds checking, e.g., the batch size is always smaller than the buffer size, which is enforced once when the driver is initialized. Such semantic guarantees are too complex for current Rust compilers and require full-featured formal methods.

Rust provides a way to suppress the bounds check using unsafe accessor functions like `get_unchecked_mut()`, which we use in several variants to isolate the cost of bounds checks:

```
1 let slot=unsafe{self.tx_slots.get_unchecked_mut(idx as usize)};
2 slot.take().expect("missing tx parked slot")
3 unsafe { *self.tx_slots.get_unchecked_mut(idx as usize)
4         = Some(new_buf.into_in_flight()) }
```

Enum types In Rust, option types commonly represent potentially nullable values using `Some(T)` and `None`. This introduces several sources of overhead: updating the enum tag, space to store the discriminant, and unwrapping an `Option<T>` (Figure 2, L7-8). First, Rust enums might require a discriminant to record the active variant. Rust can often eliminate this extra tag using a *niche* value, i.e., an invalid representation of the payload type that can be reused to encode enum variants. In particular, `Option<T>`, `Option<Box<T>>`, `Option<NonNull<T>>`, and `Option<fn()>` can be represented with the same size and alignment as the underlying pointer-like type, using null or another invalid value to represent `None` [10, 26]. In Figure 2,

```

1 let buf = pkt_buf_alloc(pool)
2 - .expect("err: pool underflow");
3 + .ok_or(DriverError::PoolUnderflow { .. })?;

```

(a) Replacing `expect()` with `ok_or()`

```

1 mov edx,DWORD PTR [rcx+0x18]
2 test edx,edx
3 - je 1c0bc // .expect() panic path
4 + je 6b794 // .ok_or() handler

```

(b) Difference in assembly code

Figure 3. Panics and explicit error checks compile to the same comparison and branch instructions

`Option::expect()` uses the loaded pointer itself as the niche to test for `None` (Figure 2b, L5 and L7-8).

The niche value optimization is not always possible. For example, types such as `Option<usize>` and `Option<u64>` generally need additional representation space because every bit pattern is a valid integer value.

Second, when the payload is moved out of an `Option<T>` that remains live, the remaining slot must still be updated to the `None` state. For example, an `Option::take()` operation would replace the reference with a `None` value in the same `Option<T>`, so subsequent accesses and destruction observe an empty slot. This manifests in an additional store operation. Our variants use `std::mem::replace()`, which immediately stores the new reference, removing the need to store `None` first.

Error handling and panic avoidance In Rust, additional overhead can stem from handling error conditions. In our example, unwrapping the `Option<T>` with `expect()` introduces two assembly instructions that check for `None` and branch to the panic handler (Figure 2, L7-8).

While process-terminating panics are acceptable in user space, they undermine the reliability of mission-critical systems such as operating system kernels. The Rust-for-Linux (RFL) guidelines explicitly forbid panics [24]. Because the Linux kernel lacks unwinding support, a Rust panic triggers a kernel `BUG()`, potentially leaving global locks held, data structures corrupted, or resources leaked. Unwinding is particularly infeasible in mixed C/Rust kernels, as C stack frames cannot be safely unwound across nested FFI boundaries—a constraint that prior language-based OS projects avoided through strict interface boundaries [20, 2, 18, 14].

Consequently, driver code must explicitly handle error paths to eliminate panics (e.g., by performing manual bounds checks or using `ok_or()` to transform `None` into explicit error types). This overhead, however, does not materialize in the error-handling path. In Figure 3, panic handling and explicit error handling with `ok_or()` (Figure 3a) generate the same comparison (Figure 3b, L2) and jump instructions, differing only in their jump targets (Figure 3b, L3 vs. L4). Although the panic- and error-handling instructions at those jump targets differ in static footprint, they are never executed at runtime.

An interesting detail is that explicit error handling can sometimes suppress inlining. Hot-path per-batch helpers like

```

1 movzx eax,WORD PTR [rbp-0xb10] // load free
2 test ax,ax // underflow check: free == 0 ?
3 je 1e2fe // -> panic on underflow
4 dec eax // free - 1
5 mov WORD PTR [rbp-0xb10],ax // store back only if safe

```

Figure 4. Overhead of the overflow check for `self.free -= 1`

`flush_rx_rearmed_tail()` and `flush_reflect_tx_batch()` were usually inlined into the caller function by the compiler. Changing the function signatures to return `Result<T>` prevented inlining. **Overflow checks** Normally, Rust does not enable integer overflow checks in release builds. Lack of overflow checking can result in complex semantic errors, e.g., an overflowing array index can remain in bounds but access a different element. Rust allows developers to enable overflow checks with the `-C overflow-checks` flag, which triggers a panic when an arithmetic operation overflows. Rust generates four additional instructions when overflow checks are enabled (Figure 4, L1-3 and L5).

Coding guidelines may require avoiding panics and using explicit APIs that check for overflow, such as `checked_add()`, `checked_sub()`, `checked_mul()`, and `checked_div()`, which return `None` if an operation overflows, underflows, or divides by zero. This allows the code to handle an overflow error but at the cost of additional runtime checks.

Interior mutability types The borrowing discipline of Rust is too restrictive for most practical programs. Rust therefore provides controlled ways of bypassing ownership rules with interior mutability types that enforce safety rules dynamically. For example, `RefCell<T>` enforces the borrow rules at runtime rather than at compile time. Types such as `Cell<T>`, while not adding runtime overhead, might suppress some compiler optimizations.

Similarly, in many cases strictly linear lifetimes are impractical. Rust uses a collection of trusted types such as `Rc<T>` and `Arc<T>` to implement dynamically reference-counted lifetimes. The wrapper type implements runtime reference counting and deallocates the object when the counter drops to zero. Atomic reference counting can be expensive when multiple cores access the counter because of the cost of moving its cache line between cores.

Validation APIs In Rust, many safe APIs validate data before constructing a type with an invariant. For example, string slicing also checks that byte offsets are valid UTF-8 character boundaries, not merely in range [10]. Once the type is constructed, later code can rely on the invariant.

Fat pointers In Rust, ordinary pointers to statically sized types are thin pointers: they contain only an address, because the compiler knows the size of the pointee from its type. Pointers become wide pointers when they point to a dynamically sized type (DST), where some part of the pointee’s layout is known only at runtime. The main size-carrying cases are pointers to slices and strings: `&[T]`, `&mut [T]`, `*const [T]`, `Box<[T]>`, `Arc<[T]>`, and similar pointer forms carry

a data address plus a length in elements. The same rule applies to structs whose final field is unsized, such as `struct Packet{header: Header, payload: [u8]}`. A pointer to `Packet` carries the metadata of the trailing payload field. Rust only allows such directly unsized fields in the final position, since otherwise the compiler could not determine the offsets of later fields. By contrast, references to arrays such as `&[T; N]` are thin because `N` is part of the type, and types such as `Vec<T>` or `String` store their length inside the object rather than in the pointer. Trait-object pointers such as `&dyn Trait` are also wide, but their metadata is a vtable rather than a per-object length. Fat pointers use two machine words instead of one, which can increase argument-passing cost, register pressure, stack usage, structure size, and cache footprint. Note that in most cases the C code would have to carry a length field along with a dynamic data structure.

Dynamic dispatch In most cases the Rust compiler can monomorphize generics like `fn f<T: Trait>(x: T){ ... }`. The compiler generates specialized assembly without vtable dispatch. Trait objects, however, use dynamic dispatch:

```
1 fn f(x: &dyn Trait) {
2     x.method(); // indirect call through vtable
3 }
```

This `&dyn Trait` is a fat pointer consisting of a data pointer and a vtable pointer. Calls through the vtable are indirect and can inhibit inlining and branch prediction.

Drop glue and destructor paths When an initialized value goes out of scope, is overwritten, or must be cleaned up on an early exit, the compiler inserts *drop glue*: code that invokes the type's user-defined destructor, if any, and then recursively destroys its fields [10]. For plain data types this code is typically optimized away, but for resource-owning abstractions it can translate into real work, such as freeing heap allocations, decrementing reference counts, releasing locks, unmapping DMA buffers, or returning packet buffers to a pool. In addition to this semantic cleanup cost, Rust may generate auxiliary control flow to ensure that destruction occurs exactly once on all paths through the program.

4 Empirical Analysis

Experiment setup We compare the C and Rust L2 forwarding implementations using minimum-sized packets to stress the per-packet critical path. Experiments run on CloudLab PowerEdge C6620 servers configured with an Intel E810 100 GbE adapter (Table 2).

For 64-byte frames, the 100 GbE line rate is 148.8 Mpps after accounting for the 20 bytes of preamble/SFD and inter-frame gap on the wire. Our experiments use a single RX/TX queue pair, and the measured peak is the single-queue software transmit capacity rather than the aggregate multi-queue capacity of the NIC.

Overhead analysis We implement several driver variants in C and Rust that vary in their use of bounds checks, overflow checks, and explicit error handling. We also implement

ixy-like variants that use unsafe pointer arithmetic on the packet pool. We evaluate these variants both with and without additional safety mechanisms.

In `C|RefPool`, packets are tracked using raw pointers to buffers in the packet pool, and these pointers are passed through the driver and descriptor queues. Free packet pointers are tracked using a stack, which increases packet reuse and reduces cache evictions. This root variant is our fastest implementation across the entire corpus (Table 1).

The `Rust|Unsafe(RefPool)` variant implements this design by defining the entire buffer pool as one long-lived mutable slice. It then consumes the slice through iteration to obtain long-lived, exclusive mutable references to individual packets, which are subsequently used as packet-buffer handles. The Rust type system ensures that the packet references remain live and have unique ownership. To isolate the impact of bounds checks, we manually suppress them with `get_unchecked()` for all array accesses in this variant. Packets are swapped out of the slot arrays using `std::mem::replace()`, and their presence is checked using `.expect()`. Although the overall per-packet instruction count changes only from 115 to 114, throughput drops by 3.67%.

`Rust|Safe(RefPool)` restores the bounds checks omitted in `Rust|Unsafe(RefPool)`. Each indexing site adds a comparison (ALU) and a conditional jump (Ctrl). Across the affected array accesses, these checks add 15 instructions, bringing the total to 129 and reducing throughput by 3.3%. To compare how bounds checks impact C code, we instrument `C|RefPool` with a manually defined bounds-check macro in `C|Safe(RefPool)`. This adds 10 instructions and reduces throughput by 1.74%.

`Rust|Safe(RefPool+Overflow)` is configured with `overflow-checks = true`. This enables integer overflow and underflow checking for five increment operations on the critical path. Two of these operations use `saturating_add()` to avoid panicking. Similarly, `C|Safe(RefPool+Overflow)` adds manual overflow checks to all the same sites. The generated instruction counts differ between the `+Overflow` variants because the compilers use different instruction sequences (`leaq` versus `test` and `jmp`). In the Rust variant, overflow checking adds 19 instructions and reduces throughput by 7.21%.

`Rust|Safe(RefPool+Errors)` replaces panic-inducing `.expect()` calls with explicit error propagation. The code checks returned values and propagates errors rather than aborting. Although this change does not affect the instruction count at the checking sites (Figure 3), it prevents functions with the `DriverError` return type from being inlined. The lack of inlining adds 18 instructions, primarily memory accesses, further reducing throughput by 10.5%.

The `PtrArith` family of drivers tracks packet buffers as `u32` indices into the packet pool array. We use packet indices throughout the code, converting each index to an actual buffer address only when placing it in a DMA descriptor. In the safe variant, bounds-checked array accesses ensure that every resolved address refers to a buffer in the packet pool.

Variant and description	THROUGHPUT		PER PACKET (+per-batch / B, B=64)							
	Gbps	Mpps	Inst	Cyc	Ld	St	Reg	ALU	Ctrl	
1. C RefPool Hand-written C, <code>struct</code> <code>pkt_buf</code> * pointers	39.60 —	82.49 —	113+2.12 —	32.1+5.18 —	30 —	18 —	14 —	42 —	12 —	
2. C Safe(RefPool) <i>(delta from 1)</i> Instrumented with bounds checks to mimic Rust’s safe indexing	38.91 (−0.69)	81.06 (−1.43)	123+2.30 (+10)	32.1+4.93	30	18	14	47 (+5)	17 (+5)	
3. C Safe(RefPool+Overflow) <i>(delta from 2)</i> __builtin_*_overflow() on hot-path arithmetic	37.35 (−1.56)	77.81 (−3.25)	131+2.48 (+8)	35.0+4.77 (+2.9)	33 (+3)	19 (+1)	13 (−1)	47	22 (+5)	
4. Rust Unsafe(RefPool) <i>(delta from 1)</i> Rust references, bounds checks suppressed with <code>get_unchecked_mut()</code>	38.14 (−1.46)	79.46 (−3.03)	112+2.31 (−1)	32.5+5.23 (+0.4)	25 (−5)	21 (+3)	15 (+1)	46 (+4)	13 (+1)	
5. Rust Safe(RefPool) <i>(delta from 4)</i> Safe Rust with normal bounds checks	36.87 (−1.27)	76.80 (−2.66)	127+2.61 (+15)	32.9+5.36 (+0.4)	29 (+4)	21 (+2)	17 (+2)	52 (+6)	17 (+4)	
6. Rust Safe(RefPool+Overflow) <i>(delta from 5)</i> Cargo <code>overflow-checks</code> = <code>true</code> , plus saturating hot-path counters	34.21 (−2.66)	71.26 (−5.54)	146+2.75 (+19)	35.5+6.21 (+2.6)	31 (+2)	22 (+1)	19 (+2)	55 (+3)	24 (+7)	
7. Rust Safe(RefPool+Errors) <i>(delta from 5)</i> Panics on the hot path replaced with <code>DriverError</code> return type handled by <code>.ok_or()</code>	33.00 (−3.87)	68.75 (−8.05)	145+3.14 (+18)	37.2+5.67 (+4.3)	37 (+8)	28 (+7)	17 (+2)	56 (+4)	20 (+3)	
8. C PtrArith <i>(delta from 1)</i> C driver with ixy-style addressing: <code>uint32</code> indices for raw pointer arithmetic	37.46 (−2.14)	78.05 (−4.44)	120+2.36 (+7)	31.9+5.84 (−0.2)	33 (+3)	20 (+2)	14 (+2)	47 (+5)	13 (+1)	
9. Rust Unsafe(PtrArith) <i>(delta from 8)</i> Rust ixy-style indices, bounds checks suppressed with <code>get_unchecked_mut()</code>	38.43 (+0.97)	80.05 (+2.00)	113+2.31 (−7)	30.6+6.17 (−1.3)	35 (+2)	16 (−4)	16 (+2)	44 (−3)	10 (−3)	
10. Rust Safe(PtrArith+Overflow) <i>(delta from 9)</i> Bounds and overflow checks enabled	34.77 (−3.66)	72.43 (−7.62)	153+2.88 (+40)	34.6+5.99 (+4.0)	41 (+6)	18 (+2)	18 (+2)	56 (+12)	28 (+18)	

Table 1. Throughput and instruction counts for C and Rust variants at batch size 64. Deltas show performance and cycle differences from the closest implementation.

Table 2. Experimental configuration of the CloudLab PowerEdge C6620 server

Component	Configuration
CPU	Intel Xeon Gold 5512U (Emerald Rapids), 28 cores, 56 threads
Frequency	2.1 GHz base, 3.0 GHz pinned
Caches	48 KiB L1d + 32 KiB L1i / core; 2 MiB L2 / core; 52.5 MiB shared L3
Memory	128 GiB DDR5 ECC (8×16 GiB at 4800 MT/s), 8 channels
NIC	Intel E810-C 2×100 GbE (PCIe 4.0 ×16)
OS	Ubuntu 22.04.2 LTS, Linux 5.15.0-177
Boot	<code>intel_iommu=on iommu=pt</code> , 8×1 GiB huge pages, THP madvise

However, the indices do not carry Rust's ownership and linearity guarantees. The driver code can confuse buffer indices and, for example, accidentally put the same index (and hence buffer) into both RX and TX queues. Notably, this variant is the closest to the one used in ixy [6]. `Rust|Unsafe(PtrArith)` implements this design with the bounds checks disabled (using the same techniques as `Rust|Unsafe(RefPool)`). Compared with `C|PtrArith`, it provides a marginal throughput improvement because the compiler calculates the pool offsets differently. The `PtrArith` addressing scheme **adds 7 instructions** to the C implementation. The Rust compiler, however, avoids this cost. In C, the compiler spills registers containing base addresses, which must then be reloaded frequently, whereas the Rust compiler retains them in registers. In Rust, the `noalias` annotation prevents these spills; however, adding the `restrict` qualifier in C had no effect. The `C|PtrArith` version **incurs a 5.4% throughput degradation**. `Rust|Safe(PtrArith+Overflow)` adds

both bounds and overflow checks. The performance is comparable to `Rust|Safe(RefPool+Overflow)`.

Performance on different batch sizes At small batch sizes, NIC latency dominates; all variants and DPDK achieve approximately 8 Mpps (Figure 5). At larger batch sizes (greater than 32), `Rust|Unsafe(PtrArith)` is 2.5–3.6% faster than `C|PtrArith`; at smaller batch sizes, it can be up to 1.5% slower. Without safety checks, Rust is typically 2–4% slower than C. With safety checks, the gap increases to 3–6% and reaches 9% when overflow checks are also enabled. Bounds checks are the least expensive checks to add, **reducing throughput by less than 4%** across all batch sizes for both C and Rust. Overflow checks alone can **reduce throughput by more than 7%** at some batch sizes in C and by **more than 10%** at several batch sizes in Rust. Explicit error handling **reduces throughput by almost 10%**, primarily because it inhibits inlining.

Instruction cost correlation study To estimate the performance cost of individual instructions, we fit a linear regression model to the measured cycles per packet at batch size 256 using per-packet dynamic instruction and store counts together with an indicator variable. We evaluate the model using leave-one-out cross-validation (LOOCV). Each fold withholds one variant, fits the model to the remaining variants, and predicts the held-out variant, with maximum errors of approximately 0.3 cycles per packet.

Instruction count alone is insufficient; the model also accounts for stores and a `PtrArith`-specific fixed cost:

$$\text{cyc/pkt} = 20.7 + 0.063 I + 0.283 S + 2.33 X \quad (1)$$

Here, I is the instruction count, S is the store-instruction count, and X is an indicator equal to 1 for `PtrArith` variants and 0 otherwise. The indicator captures a modeled fixed cost of 2.33 cycles per packet for `PtrArith` variants. The model achieves $R^2 = 0.996$; LOOCV yields a typical error of 0.19

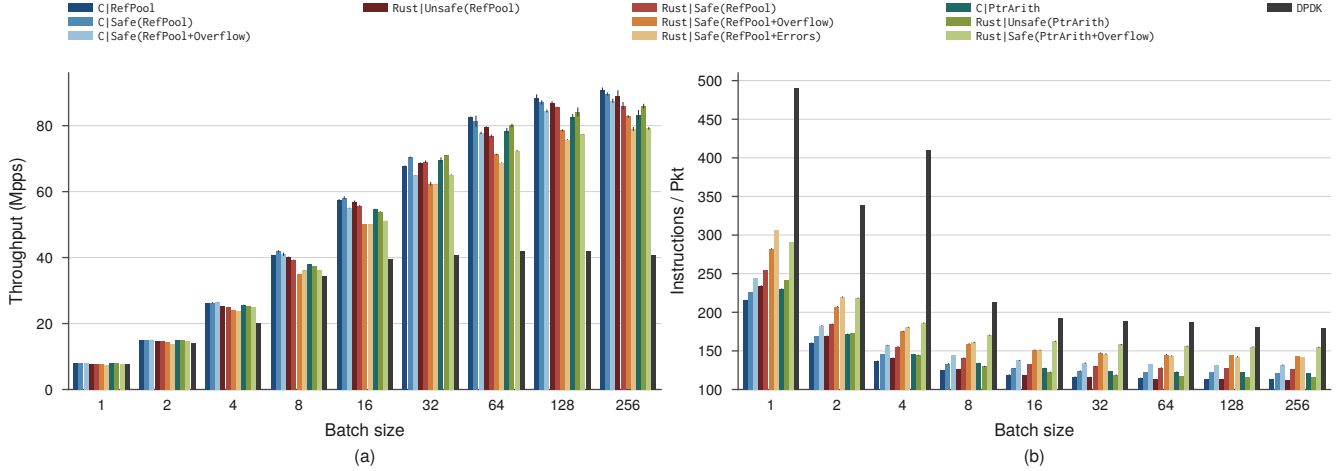


Figure 5. Throughput and profiled critical-path instruction count across batch sizes

cycles per packet ($\epsilon_{\max} = 0.31$). An independent replication using nine variants recovers all coefficients within 10%.

Separate instruction-class fits estimate that each store costs approximately 0.35 cycles per packet, whereas each load costs approximately 0.1 cycles per packet, no more than an average instruction. These fits also show that the loads and branches added by bounds checks have a negligible marginal effect on performance.

Predicting performance on more complex code To gain a deeper understanding of store costs, we experimented with more complex Rust implementations that add code to the driver’s critical path. Specifically, we added a metadata array that tracks each buffer’s I/O virtual address and size. We also widened the buffer reference to 16 bytes to store a 4-byte index alongside the 8-byte buffer-pool base, with 4 bytes of compiler-added padding. This more complex representation adds stores to the critical path, increasing store-buffer pressure and raising the estimated cost of each store to approximately 0.73 cycles per packet. To test this hypothesis, we use synthetic experiments that independently vary the numbers of arithmetic, control-flow, store, and load instructions. For the metadata-array design, the resulting regression model is:

$$\text{cyc/pkt} = 16.38 + 0.071 I + 0.66 S \quad (2)$$

Because these synthetic experiments hold the algorithm fixed, the model does not require an indicator analogous to X in Equation 1. The model achieves $R^2 = 0.997$, with LOOCV yielding a typical error of 0.57 cycles per packet ($\epsilon_{\max} = 0.61$).

Together, the instruction and store coefficients assign each store a marginal cost of approximately 0.72 cycles on the per-packet path. Separate instruction-class fits estimate a cost of 0.18 cycles per load. Hardware counters additionally identify store-buffer saturation as the likely cause. The simpler `RefPool` design remains below this limit, resulting in a weaker relationship between store count and execution time.

5 Artifacts and Reproducibility

All of our variants and the scripts used for benchmarking, instrumentation, as well as parsing the instrumentation data and disassemblies are made available as a publicly hosted GitHub repository (<https://github.com/mars-research/userspace-ice>).

A combination of GPT-5.6 and Claude Opus 5 was used to aid the development of both the C and Rust variants. All reported data was manually inspected and reproduced. A Nix environment is included to reproduce the exact toolchains used to compile the variants (clang 19.1.7 / rustc 1.86 with the same LLVM on Ubuntu 22.04). Perf 6.12.63 and Pin 4.3 were used in combination for the instrumentation. Additional information is available in the `README.md`.

6 Conclusions

The growing popularity of Rust raises a natural question: What performance cost does Rust’s safety impose? Our findings are cautiously optimistic. Lean, carefully crafted safe Rust code can approach the performance of unsafe C in demanding, cycle-optimized systems workloads. However, even modest representation changes, such as widening a pointer, and additional safety features, such as overflow checks and explicit error handling, can introduce measurable overhead. Closing the remaining performance gap between safe Rust and C will likely require compiler improvements and architectural support that make dynamic checks, richer data representations, and safety-related state encoded in types such as `Option<T>` cheaper on the fast path.

Acknowledgments

We thank the PLOS '26 reviewers for helping us improve this work and the CloudLab team, especially Mike Hibler, for their generous help in accommodating our numerous requests. This research was supported in part by the National Science Foundation under grants 2239615 and 2220410.

References

- [1] Apache DataFusion Authors. Apache DataFusion. 2026.
- [2] Kevin Boos, Namitha Liyanage, Ramla Ijaz, and Lin Zhong. Theseus: An experiment in operating system structure and state management. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.
- [3] Bytecode Alliance. Wasmtime. 2026.
- [4] Cloud Hypervisor Authors. Cloud Hypervisor. 2026.
- [5] Cloudflare. Open sourcing Pingora: our Rust framework for building programmable network services. February 28, 2024.
- [6] Paul Emmerich, Simon Ellmann, Fabian Bonk, Alex Egger, Esaú García Sánchez-Torija, Thomas Günzel, Sebastian Di Luzio, Alexandru Obada, Maximilian Stadlmeier, Sebastian Voit, and Georg Carle. The case for writing network drivers in high-level programming languages. In *Proceedings of the ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, 2019.
- [7] Firecracker Authors. Firecracker: secure and fast microVMs for serverless computing. 2026.
- [8] Diane Hosfelt. Implications of rewriting a browser component in Rust. February 28, 2019.
- [9] Kata Containers Authors. Kata Containers runtime-rs. 2026.
- [10] Steve Klabnik, Carol Nichols, and Chris Krycho. *The Rust Programming Language*. 2026.
- [11] Hongyu Li, Liwei Guo, Yexuan Yang, Shangguang Wang, and Mengwei Xu. An empirical study of Rust-for-Linux: The success, dissatisfaction, and compromise. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC)*, 2024.
- [12] Zhaofeng Li, Vikram Narayanan, Xiangdong Chen, Jerry Zhang, and Anton Burtsev. Rust for Linux: Understanding the security impact of Rust in the Linux kernel. In *Proceedings of the Annual Computer Security Applications Conference (ACSAC)*, 2024.
- [13] Zhaofeng Li, Jerry Zhang, Joshua Tlatempa-Agustin, Xiangdong Chen, and Anton Burtsev. Understanding the security impact of CHERI on the operating system kernel. In *Proceedings of the Annual Computer Security Applications Conference (ACSAC)*, 2025.
- [14] Yao Liu, Jialun Li, Tobin Feldman-Fitzthum, Stephen Crane, Dan Williams, Tycho Andersen, Guofeng Zhang, and Lin Zhong. Safe and usable kernel extensions with Rex. February 26, 2025.
- [15] Microsoft. OpenVMM and OpenHCL. 2026.
- [16] Microsoft Open Source. Introducing Hyperlight: virtual machine-based security for functions at scale. November 7, 2024.
- [17] MSRC Team. We need a safer systems programming language. July 18, 2019.
- [18] Vikram Narayanan, Tianjiao Huang, David Detweiler, Dan Appel, Zhaofeng Li, Gerd Zellweger, and Anton Burtsev. RedLeaf: Isolation and communication in a safe operating system. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.
- [19] Nicholas Nethercote et al. The Rust performance book. November 2020.
- [20] Aurojit Panda, Sangjin Han, Keon Jang, Melvin Walls, Sylvia Ratnasamy, and Scott Shenker. NetBricks: Taking the V out of NFV. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016.
- [21] rustls Contributors. rustls: a modern TLS library written in Rust. 2026.
- [22] The ChromiumOS Authors. crosvm: the Chrome OS virtual machine monitor. 2026.
- [23] The FreeBSD Foundation. 2024 FreeBSD developer summit: integration with Rust. June 27, 2024.
- [24] The Linux Kernel Developers. Coding guidelines. 2026.
- [25] The Linux Kernel Developers. Rust. 2026.
- [26] The Rust Project Developers. repr(Rust). 2026.
- [27] Windows Insider Team. Announcing Windows 11 Insider Preview Build 25905. July 12, 2023.
- [28] Yuchen Zhang, Yunhang Zhang, Georgios Portokalidis, and Jun Xu. Towards understanding the runtime performance of Rust. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022.